

Evaluation of YOLO-Based Models with CBAM and Backbone Modifications for Robust Underwater Debris Detection

Manish S R, V. Sreevanth, Dr. Rimjhim Padam Singh

Department of Computer Science and Engineering

Amrita School of Computing, Bengaluru

Amrita Vishwa Vidyapeetham, India

bl.en.u4cse22127@bl.students.amrita.edu, bl.en.u4cse22176@bl.students.amrita.edu,

ps_rimjhim@blr.amrita.edu

Abstract—Marine debris is a major threat to aquatic ecosystems, which requires efficient and effective detection methods for detection, preparedness, and environment monitoring. Here we present a deep learning method for underwater litter detection and quantify performance using variants of the YOLO (You Only Look Once) object detection framework. Within this analysis, fourteen YOLO models were trained with the TrashCan dataset, using standard baseline models as well as models with CBAM attention, and backbone modification such as ResNet or YOLOv12n. The YOLOv10 and YOLOv11 architectures were also modified to optimize performance in difficult underwater environments, such as turbidity, low visibility, and image distortion. Using standard object detection metrics precision, recall, and mean Average Precision (mAP) to evaluate model performance, the proposed pipeline provided powerful and scalable real-time information on below water litter detection, and promise to aid in automated ecological monitoring for the persistence of marine ecosystems.

Index Terms—Underwater debris detection, YOLO, deep learning, object detection, CBAM, ResNet, environmental monitoring, convolutional neural networks.

I. INTRODUCTION

Marine ecosystems are increasingly threatened by anthropogenic debris including plastics, metals, and textiles. Anthropogenic debris threatens marine biodiversity and food security and poses significant risks to maritime businesses and coastal communities and economies that depend on healthy ocean resources. Unchecked anthropogenic debris changes and disrupts habitat, reversible or irreversible habitat alterations, and pushes to maybe unsustainable use of marine coastal biodiversity. Identifying and classifying this debris is a crucial first step to enable clean-up and protective efforts for marine biodiversity.

Unfortunately, underwater object detection is inherently difficult because of the underwater environment, including water turbidity, ambient illumination, motion blur, and visual disturbances. Using traditional monitoring techniques, such as diver-based monitoring and remotely-operated underwater vehicles (ROV), are labour-intensive, expensive, and have limited scalability. The challenges presented by using human divers or ROVs want to highlight the importance of developing

automated and real-time detection systems that can operate autonomously and consistently in underwater environments.

This research proposes a deep learning-based approach to automate marine debris identification using the YOLO (You Only Look Once) object detection framework. YOLO has demonstrated comparable precision and a significant speed advantage over other object detection strategies which may facilitate applications requiring real-time approach and identification of marine debris. The overall performance of small (YOLOv8n) to extra small (YOLOv12s) YOLO options, as well as attention-based (CBAM), and hybrid backbone variant, were studied. In addition, simple architectural modifications were made such as incorporating CBAM reasoning and utilizing residual learning techniques, to allow for a richer feature representation with low contrast and/or highly occluded, cluttered, underwater imagery.

The models were trained and tested using the TrashCan dataset which consisted of labelled images depicting different types of submerged waste labels and was created in accordance with Roboflow standards. Model performance was measured based on well known object detection metrics namely, precision, recall, and mean Average Precision (mAP) at an intersection over union (IoU) score of 0.5. A comparative analysis of baseline, attention-based, and backbone-based YOLO models showed a positive relationship between both, attention-based and backbone modification for small to extra small marine debris detection.

This research offers a cost-effective and scalable solution for underwater litter detection that can be applied to real-time monitoring systems and autonomous underwater vehicles. This work contributes to marine conservation efforts by providing a reliable tool for environmental monitoring and litter reduction.

This paper is organized as follows. Section I presents the Introduction. Section II reviews the related literature. Section III outlines the methodology adopted in this study. Section IV describes the implementation details of the proposed models. Section V presents the experimental results. Section VI provides a comparative analysis of the model performances. Finally, Section VII concludes the paper and Section VIII lists the references.

II. LITERATURE REVIEW

Faiza et al. [1]proposes an optimized YOLOv8 model to detect underwater litter. Optimization is performed by fine-tuning the hyperparameters with the OFAT technique. The precision was 98.8%, which makes it very effective in underwater environments with complex object types. However, it highly relies on big datasets and fine-tuning expertise, which can be a constraint on its adaptability to new environments. Fan et al. [2]explains the AASS-a low-cost scanner-in conjunction with super resolution reconstruction techniques and an advanced YOLOv8 model for detecting litter underwater. One advantage is cost and given reduced visibility, improved detection is quite useful. However, reliance on SRR to clear images may mean a computational overhead, while environmental aspects such as turbidity remain difficult.

B et al. [3]Using architectures such as VGG19, ResNet50, and DenseNet121 on the JAMSTEC dataset, the models achieved 75% reduction in parameters while maintaining a classification accuracy of 70%. The advantages include reduced computational load and suitability for platforms with limited resources. However, this approach sacrifices a portion of accuracy for efficiency and may not perform well on complicated or diverse datasets. N et al. [4]implemented a YOLOv8 deep learning model augmented with an Efficient Channel Attention (ECA) module and Contrast Limited Adaptive Histogram Equalization (CLAHE) for marine plastic detection. The approach enhanced detection accuracy and real-time processing with an F1-score of 97.8% and mAP of 95.4%. Advantages include enhanced clarity of images taken in adverse underwater conditions as well as superior performance over previous YOLO versions. The method, however, tends to consume a lot of computational resources, particularly in large-scale applications.

Bushra et al. [5]Comparisons were made between the CNN and transformer architectures on the classification of underwater debris by a dataset from the Japan Agency for Marine-Earth Science and Technology. The Swin transformer obtained the highest accuracy of 93.08% over DenseNet121 (90.99%) and MobileNetV2 (84.91%). The transformers performed superiorly in accuracy while being much more resource-intensive compared to the CNNs in terms of the number of parameters. Sidharth et al. [6]This study relies on the YOLOv8n model trained on the Trashcan dataset to detect underwater debris, achieving an mAP of 86.01%, thus surpassing other YOLO versions and R-CNN-based models. Advantages include lightweight design, plus superior recall at 94.6% in real-time applications. Still, its comparatively lower mAP than state-of-the-art models shows that there is still some room for improvement in challenging environments.

Prabhavathy et al. [7]This paper proposes the AIT-YOLOv7 model, a modified version of YOLOv7 which integrates Modified Swin Transformer Blocks, CBAM, and diffusion models for better quality and detection in underwater images. This model outperformed YOLOv3, YOLOv4, and so on in achieving a mean accuracy of 81.4% in its classifications

under the TrashCan dataset. Strengths: clearer images and better detection precision. But the complex nature of this model may be a detraction for deploying it to resource-limited systems. Ritik et al. [8]The analysis carried out on the datasets includes Mask R-CNN, YOLOv8, EfficientDet-D0, and YOLACT-the last detected mAP is 71.4% and is the fastest in training speed. Pros include YOLOv8 for real-time detection and high-performance efficiency; it is also simple to use when objects are detected in small scenes. Conversely, the disadvantage of YOLOv8 has to do with its poor performance in the detection of small or occluded objects. Brendan et al. [9]assessed the use of Mask R-CNN and YOLACT in segmenting underwater litter with mean average precisions of 0.377 and 0.365, respectively. YOLACT had a higher speed useful for real-time applications, while Mask R-CNN provided more accuracy for further analysis. The lightweight make of YOLACT allows for broader deployment, but both models struggled with small or elaborate underwater litter. Longyi et al. [10]study proposes the GST-YOLO, a new and enhanced model based on YOLOv8 to be optimized utilizing GhostConv, GTR modules, and SimAM attention. The model achieved an overall accuracy of 95.4% with a 51% reduction in size and 54% in computational efficiency. Advantages include the ability to deploy on resource-constrained devices and improved feature extraction. On the other hand, it does depend on a specific dataset that might constrain its generalization.

Khriss et al. [11]Faster R-CNN, SSD, YOLOv8, and YOLOv9 was employed on TrashCAN and DeepTrash datasets for underwater litter detection. YOLOv9 was found to outrun the other task-solving approaches by far, evidencing superior precision, recall, and mAP. It is an effective solution however, high computational requirements might restrict its use in real-time. Walia et al. [12]conducted an evaluation of deep learning frameworks intended for underwater trash detection and localization. They identified problematic aspects such as image distortion and data scarcity and examined baseline algorithms for possible implementation on AUVs. Their study indicates that there is a need for robust solutions, yet there also exist limitations to current datasets and algorithm performance metrics.

Zhu et al. [13]proposed YOLOv8-C2f-Faster-EMA, which improves upon YOLOv8 with optimized backbone layers and attention mechanisms used to detect underwater garbage. This model showed great improvement in precision and mAP while the advancements with respect to optimization might stand as a challenge for deployment. Zhang et al. [14]pioneered YOLOv7t-CEBC, another version upgraded from YOLOv7-tiny, which is minted to improve default underwater debris detection. It operates at 81.8% mAP with great frame rates of 118 FPS; therefore, its performance is better in real-time scenarios. However, detection of very small and camouflaged objects generates prospect challenges.

Gupta et al. [15]developed a two-stage YOLOv8 model with an underwater image enhancer (UIE) and hybrid feature extractor (HEFA), achieving an accuracy of 98.5%, facing thus scalability issues in complex underwater environments. Demir

et al. [16] Using ResNet101 with Neighborhood Component Analysis (NCA) and kNN, the author classified underwater debris and marine life using a hybrid dataset. The model achieved 99.35% accuracy; however, real-time applicability is left wanting.

Alsawaylimi et al. [17] evaluates YOLOv8-Seg for instance segmentation of underwater trash. It displayed admirable precision and inference times but faced problems of detecting small objects. Jiang et al. [18] optimized YOLOv8 with CloFormer and SCConv modules for marine debris detection. The model achieved a high gain in mAP and F1 scores but experienced challenges in generalization into various underwater scenarios.

Sánchez et al. [19] worked on detecting marine debris utilizing Mask R-CNN on the CleanSea dataset, where the division of synthetic marine scenes intended to solve the problem of lack of data. While it achieved state-of-the-art results, the reliance on synthetic data would limit its usage in real life. Deng et al. [20] introduced a combination of dilated convolutions with spatial-channel attention to Mask R-CNN for marine debris detection, resulting in an increased mAP score but is computationally expensive, which poses additional challenges for deploying it on resource-limited hardware.

III. PROPOSED METHODOLOGY

A. Baseline YOLO Model Training

Several YOLO versions were trained on the baseline dataset, including YOLOv8n, YOLOv9c, YOLOv10n, YOLOv10m, YOLOv11n, YOLOv11s, YOLOv11m, YOLOv12n, and YOLOv12s, which were considered for benchmarking against further enhancement presented in this work. Ultralytics YOLO framework implementation supported the parallel training on GPU platform, also running in Colab and Kaggle. Standard hyperparameters across the standard setups included 50 epochs and 16 batch size with image size fixed at 416×416. This setup allowed a straightforward, unbiased assessment of the models' performance across various versions.

B. Attention-Augmented YOLO Model

Attention mechanisms were added into important points of the network in the YOLOv11 architecture to improve detection accuracy for small or occluded objects. Concretely, CBAM (Convolutional Block Attention Modules) were added in three places: after layers 14, 18, and 22 within the head (neck path), focusing on enhancing attention across features at different scales. Optionally, the C2PSA (Cross-Stage Partial Spatial Attention) could be added in the backbone at layer 10, giving spatially aware attention to help strengthen feature representation at lower layers of the backbone. These custom modules were implemented as PyTorch modules fully integrated within the Ultralytics YOLO framework to encourage prioritization of features as well as more robust detection output.

C. Custom Backbone YOLO Models

Beyond attention modules, this work also investigates the use of custom CNN backbones to even improve YOLO's feature extraction. We used a lightweight ResNet-style backbone

in some models as a replacement for the traditional YOLO convolutional backbone. The ResNet-style feature learning was used in the YOLOv11n version by changing the early features to accept features through residual learning. This approach was intended to create spatial feature representation, along with robustness to occlusion and scale variability as it pertained to smaller and more complicated objects. The approach of integrating a completely new architecture was assessed through ablation study experiments and model comparisons using side-by-side processing, ultimately showing tangible improvements in terms of detection performance.

D. Training and Evaluation Workflow

The last step of the methodology was training all the chosen YOLO model variants on the annotated and curated dataset and then performance testing with common object detection metrics like precision, recall, and mean average precision at an IoU threshold of 0.5 (mAP@0.5). In parallel to the quantitative assessment, a qualitative analysis was also conducted through visually examining the detection results in order to evaluate the accuracy and confidence of the bounding box predictions. This two-mode assessment allowed for an overall comparison across the 9 different model configurations, which reflected the effects of attention mechanisms and bespoke backbone integrations on detection precision, particularly for hard object cases.

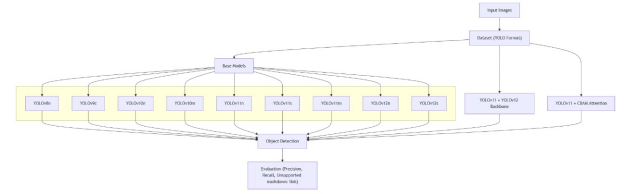


Fig. 1. System Architecture

The full system pipeline is illustrated in Fig.1 The pipeline begins with raw images as input from custom object detection datasets. These images were annotated using Roboflow, and converted into a format suited for the Ultralytics YOLO framework. After the preprocessing steps were completed, the dataset was employed for training several YOLO based architectures - baseline, attention-augmented (CBAM, C2PSA), and ResNet-layer augmented. After training models, bounding box coordinates and class labels for more than one object category were produced by the models. The predictions were then evaluated based on precision, recall, and mAP@0.5 to obtain a relative understanding of the architectural advancements.

IV. IMPLEMENTATION

The Underwater trash detection system was implemented using the Ultralytics YOLO framework developed on PyTorch. This chapter presents the steps taken for training, adapting, and

testing a set of nine YOLO model variants. All experiments were performed on GPU-based environments, such as Google Colab and Kaggle, in order to efficiently train and make inferences.

A. Dataset Description

The dataset *TrashCan* used in this study includes 7,212 annotated underwater photos that have been resized to 416×416 pixels. The photos depict a variety of underwater scenes containing trash objects, ROVs, and marine organisms (animals and plants), and are annotated with instance segmentation masks indicating the exact pixel region occupied by each object instance. The TrashCan dataset is sourced from *J-EDI* (JAMSTEC E-Library of Deep-sea Images), which is a curated dataset collected by the Japan Agency for Marine-Earth Science and Technology (JAMSTEC).

The dataset includes three major object categories with 22 distinct classes spanning both biological and anthropogenic items: animal_crab, animal_eel, animal_etc, animal_fish, animal_shells, animal_starfish, plant, rov, trash_bag, trash_bottle, trash_branch, trash_can, trash_clothing, trash_container, trash_cup, trash_net, trash_pipe, trash_rope, trash_snack_wrapper, trash_tarp, trash_unknown_instance, and trash_wreckage. Sample images from the training, testing, and validation splits of the dataset are shown in Fig.2, Fig.3 and Fig.4.



Fig. 2. Sample Image from Train Dataset



Fig. 3. Sample Image from Test Dataset



Fig. 4. Sample Image from Validation Dataset

The dataset's diversity and level of annotation detail make it ideal for benchmarking deep learning models for object detection and marine debris monitoring.

B. YOLO Baseline Models

To lay the groundwork for performance benchmarking, a variety of baseline object detection models were selected from various YOLO (You Only Look Once) series. The architectural choice included YOLOv8n, YOLOv9c, YOLOv10n, YOLOv10m, YOLOv11n, YOLOv11s, YOLOv11m, YOLOv12n, and YOLOv12s. These models belong to different generations of the YOLO framework, the development of which has seen successive architectural, computational, and detection improvements. The baseline models were trained together under uniform experimental conditions with the Ultralytics YOLO framework implemented in PyTorch. The images were resized to a fixed 416×416 input resolution, with each model trained for 50 epochs and a batch size of 16. A consistent configuration for training was necessary to obtain an unbiased evaluation of the capability of the respective model without being influenced by differing hyperparameters. These baseline models were used as controls in seeing how the architectural improvement- from the employment of attention modules to the modifications of custom-designed backbones- worked by testing models were pertinent in the recognition of different types of marine debris and biological organisms contained in the TrashCan dataset.

C. CBAM-Attention YOLO Model

	from	n	params	module	arguments
0	-1	1	464	ultralytics.nn.modules.conv.Conv	[3, 16, 3, 2]
1	-1	1	4672	ultralytics.nn.modules.conv.Conv	[16, 32, 3, 2]
2	-1	1	53824	ultralytics.nn.modules.block.C3k2	[32, 64, 1, False, 1.0]
3	-1	1	36992	ultralytics.nn.modules.conv.Conv	[64, 64, 3, 2]
4	-1	1	214144	ultralytics.nn.modules.block.C3k2	[64, 128, 1, False, 1.0]
5	-1	1	147712	ultralytics.nn.modules.conv.Conv	[128, 128, 3, 2]
6	-1	1	263936	ultralytics.nn.modules.block.C3k2	[128, 128, 1, True, 1.0]
7	-1	1	295424	ultralytics.nn.modules.conv.Conv	[128, 256, 3, 2]
8	-1	1	1052160	ultralytics.nn.modules.block.C3k2	[256, 256, 1, True, 1.0]
9	-1	1	164608	ultralytics.nn.modules.block.SPPF	[256, 256, 5]
10	-1	1	249728	ultralytics.nn.modules.block.C2PSA	[256, 256, 1]
11	-1	1	0	torch.nn.modules.upsampling.Upsample	[None, 2, 'nearest']
12	[-1, 6]	1	0	ultralytics.nn.modules.Concat	[1]
13	-1	1	296064	ultralytics.nn.modules.block.C3k2	[384, 128, 1, False, 1.0]
14	-1	1	2146	ultralytics.nn.modules.conv.CBAM	[128]
15	-1	1	0	torch.nn.modules.upsampling.Upsample	[None, 2, 'nearest']
16	[-1, 4]	1	0	ultralytics.nn.modules.Concat	[1]
17	-1	1	82496	ultralytics.nn.modules.block.C3k2	[256, 64, 1, False, 1.0]
18	-1	1	610	ultralytics.nn.modules.conv.CBAM	[64]
19	-1	1	36992	ultralytics.nn.modules.conv.Conv	[64, 64, 3, 2]
20	[-1, 13]	1	0	ultralytics.nn.modules.Concat	[1]
21	-1	1	246912	ultralytics.nn.modules.block.C3k2	[192, 128, 1, False, 1.0]
22	-1	1	2146	ultralytics.nn.modules.conv.CBAM	[128]
23	-1	1	147712	ultralytics.nn.modules.conv.Conv	[128, 128, 3, 2]
24	[-1, 10]	1	0	ultralytics.nn.modules.Concat	[1]
25	-1	1	1117696	ultralytics.nn.modules.block.C3k2	[384, 256, 1, True, 1.0]
26	[18, 22, 25]	1	434962	ultralytics.nn.modules.head.Detect	[22, [64, 128, 256]]

YOLOv11 summary: 202 layers, 4,851,400 parameters, 4,851,384 gradients, 14.8 GFLOPs

Fig. 5. YOLOv11n architecture with CBAM attention integration

The idea here was to assist the model in focusing more on meaningful areas of an image and to improve the discrimination of features across various scales by embedding a CBAM module into the architecture of the YOLOv11n. CBAM is a simple and lightweight attention mechanism consisting sequentially of channel attention and spatial attention, so as to allow a network to decide dynamically "what" should be focused on and "where" in an image. This is especially vital in underwater object detection where there is visual clutter and partial occlusion. In this version of YOLOv11n-CBAM, modules of CBAM are strategically inserted into the network's head at layers 14, 18, and 22. These correspond to stages in the neck path where multiscale features are aggregated just before performing prediction. Such integration is intended to improve contextual understanding and localization accuracy by manipulating refined feature maps prior to provision to the detection head.

Each CBAM module was instantiated with custom PyTorch code that was initialized with parameters like input channel size, reduction ratio, and kernel size for spatial attention maps. Such a modular design allowed the integration of this module with the Ultralytics YOLO framework, while still leaving parallel training courses intact. The model log in Fig.5 depicts instances of CBAM modules being created and connected, specifying their initialization with these hyperparameters. This adjustment enabled the YOLOv11n model to more closely pay attention to informative features in intricate underwater scenes, with resulting enhanced detection performance. The validity of this attention-assisted model was subsequently tested with quantitative measures as well as qualitative visual checks.

D. ResNet-Backbone YOLO Models

To assess how backbone structure influences detection accuracy, a ResNet-like convolutional backbone was used in the architecture of the YOLOv11n model instead of the standard backbone structure typically used in the YOLO family. The authors first introduced residual learning elements from the ResNet structure into the YOLOv11n network in the early layers only, with the neck and detection head unchanged. The primary motivation for this architectural design choice was to improve spatial representation of features and to enhance deeper semantic learning in the initial layers of the networks. The remodeled backbone promoted better gradient flow by using identity-based shortcut connections between convolutional layers and helped the model to retain more information from one layer to the next. All of these contributions are particularly helpful when detecting marine debris when they are small, occluded or visually obscured.

	from	n	params	module	arguments
0	-1	1	4856	ultralytics.nn.modules.conv.Conv	[5, 64, 3, 2]
1	-1	1	36992	ultralytics.nn.modules.conv.Conv	[64, 64, 3, 1]
2	-1	1	208832	ultralytics.nn.modules.block.ResNetLayer	[64, 64, 1, True]
3	-1	1	816540	ultralytics.nn.modules.block.ResNetLayer	[64, 128, 2, False, 3]
4	-1	1	4864000	ultralytics.nn.modules.block.ResNetLayer	[512, 256, 2, False, 4]
5	-1	1	28352512	ultralytics.nn.modules.block.ResNetLayer	[1024, 512, 2, False, 6]
6	-1	1	59813808	ultralytics.nn.modules.block.ResNetLayer	[2048, 1024, 2, False, 3]
7	-1	1	16783368	ultralytics.nn.modules.block.SPP	[4096, 1024, 5]
8	-1	1	7346176	ultralytics.nn.modules.block.C2f	[1024, 1024, 1]
9	-1	1	0	torch.nn.modules.upsampling.Upsample	[None, 2, 'nearest']
10	[-1, 5]	1	0	ultralytics.nn.modules.conv.Concat	[1]
11	-1	1	2558720	ultralytics.nn.modules.block.C3k2	[3072, 512, 1, False]
12	-1	1	0	torch.nn.modules.upsampling.Upsample	[None, 2, 'nearest']
13	[-1, 4]	1	0	ultralytics.nn.modules.conv.Concat	[1]
14	-1	1	640384	ultralytics.nn.modules.block.C3k2	[1536, 256, 1, False]
15	-1	1	590336	ultralytics.nn.modules.conv.Conv	[256, 256, 3, 2]
16	[-1, 11]	1	0	ultralytics.nn.modules.conv.Concat	[1]
17	-1	1	1379872	ultralytics.nn.modules.block.C3k2	[768, 512, 1, False]
18	-1	1	2360320	ultralytics.nn.modules.conv.Conv	[512, 512, 3, 2]
19	[-1, 8]	1	0	ultralytics.nn.modules.conv.Concat	[1]
20	-1	1	6037504	ultralytics.nn.modules.block.C3k2	[1536, 1024, 1, True]
21	[14, 17, 20]	1	1859602	ultralytics.nn.modules.head.Detect	[22, [256, 512, 1024]]

YOLOv11_resnet summary: 256 layers, 133,642,194 parameters, 133,642,178 gradients

Fig. 6. YOLOv11n model architecture with integrated ResNet backbone

The YOLOv11n model with ResNet backbone was trained and fine-tuned on the TrashCan dataset, which consists of various marine waste and biological classes. The residual learning in the backbone architecture allowed for better convergence during training and was also more precise when it came to detecting fine-grained features around object boundaries. Fig.6 shows the architecture by overview, and the improved features extraction pathway including the residual backbone.

E. YOLOv11n with YOLOv12n Backbone

To assess whether architectural inheritance from the newer model variants really helps, the YOLOv11n's backbone was swapped out for the YOLOv12n. This alteration was to provide the earlier network stages with a stronger representation, which comes from superior feature extractors endowed with state-of-the-art convolutional blocks and a deeper, more well-engineered structure: while the YOLOv11n detection head and neck were maintained, the introduction of the YOLOv12n backbone allowed for more powerful representation mechanisms into the earlier stages of the network.

	from	n	params	module	arguments
0	-1	1	464	ultralytics.nn.modules.conv.Conv	[3, 16, 3, 2]
1	-1	1	4672	ultralytics.nn.modules.conv.Conv	[16, 32, 3, 2]
2	-1	1	6540	ultralytics.nn.modules.block.C3k2	[32, 64, 1, False, 0.25]
3	-1	1	36992	ultralytics.nn.modules.conv.Conv	[64, 64, 3, 2]
4	-1	1	26080	ultralytics.nn.modules.block.C3k2	[64, 128, 1, False, 0.25]
5	-1	1	147712	ultralytics.nn.modules.conv.Conv	[128, 128, 3, 2]
6	-1	2	180864	ultralytics.nn.modules.block.A2C2f	[128, 128, 2, True, 4]
7	-1	1	295424	ultralytics.nn.modules.conv.Conv	[128, 256, 3, 2]
8	-1	2	689408	ultralytics.nn.modules.block.A2C2f	[256, 256, 2, True, 1]
9	-1	1	0	torch.nn.modules.upsampling.Upsample	[None, 2, 'nearest']
10	[-1, 6]	1	0	ultralytics.nn.modules.conv.Concat	[1]
11	-1	1	86912	ultralytics.nn.modules.block.A2C2f	[384, 128, 1, False, -1]
12	-1	1	0	torch.nn.modules.upsampling.Upsample	[None, 2, 'nearest']
13	[-1, 4]	1	0	ultralytics.nn.modules.conv.Concat	[1]
14	-1	1	24000	ultralytics.nn.modules.block.A2C2f	[256, 64, 1, False, -1]
15	-1	1	36992	ultralytics.nn.modules.conv.Conv	[64, 64, 3, 2]
16	[-1, 11]	1	0	ultralytics.nn.modules.conv.Concat	[1]
17	-1	1	74624	ultralytics.nn.modules.block.A2C2f	[192, 128, 1, False, -1]
18	-1	1	147712	ultralytics.nn.modules.conv.Conv	[128, 128, 3, 2]
19	[-1, 8]	1	0	ultralytics.nn.modules.conv.Concat	[1]
20	-1	1	378880	ultralytics.nn.modules.block.C3k2	[384, 256, 1, True]
21	[14, 17, 20]	1	434962	ultralytics.nn.modules.head.Detect	[22, [64, 128, 256]]

Fig. 7. YOLOv11n model architecture with integrated YOLOv12n backbone

The focus of this consideration is feature encoding layers to be able to capture richer spatial-semantic information and the faster approach, having the benefits of: architecturally refined by a newer YOLO version, better gradient propagation with deeper layers, and enhanced multi-scale feature fusion. These become highly useful for underwater imagery, detecting small, occluded, or cluttered marine waste objects. The modified YOLOv11n model, with YOLOv12n backbone in hand, was further trained on TrashCan dataset and evaluated against baseline and attention-enhanced variants for comparative analysis. The model log in Fig.7 illustrates the integration of the YOLOv12n backbone into the YOLOv11n framework,

showing the initialization and connection of layers adapted from the enhanced feature extraction pipeline. This hybrid configuration yielded higher detection accuracy and robustness, hence validating backbone transfer as a targeted means of improving model performance without interfering with the existing detection pipeline.

V. RESULTS

A. Performance Evaluation of Base Models

We have compared eight different models for object detection, with the image size (px) 416x416. Their performance over 50 epochs was assessed using precision, recall, and mean average precision (mAP) at different IoU thresholds. The models perceived that they had different goodnesses and variances depending on capabilities concerning objective localization and detection. A few achieved high precision but failed to show an adequate balance of precision and recall, which substantially affected their overall mAP scores.

YOLOv8n achieved Precision: 0.8939, Recall: 0.8874, mAP@50: 0.9364, and mAP@50-95: 0.7550, indicating strong object detection and localization accuracy. The YOLOv8n model's training loss and performance metrics over epochs are illustrated in Fig. 8.

YOLOv9c achieved Precision: 0.8866, Recall: 0.8472, mAP@50: 0.9086, and mAP@50-95: 0.7068, indicating better localization while maintaining a balance between precision and recall. The YOLOv9c model's training loss and performance metrics over epochs are illustrated in Fig. 9.

YOLOv9s followed similar trends, and its training metrics are shown in Fig. 10.

YOLOv10n and YOLOv11n followed similar trends, with varying degrees of performance between precision, recall, and mAP values, reflecting differences in their detection robustness and generalizability. The YOLOv10n and YOLOv11n model's training loss and performance metrics over epochs are illustrated in Fig. 11 and Fig. 12.

The YOLOv11s model achieved a notable precision of 0.9058, recall of 0.8842, mAP@50 of 0.9254, and mAP@50-95 of 0.7424. These results indicate a precision and recall performance that was high and consistent. Fig. 13 shows the training metrics.

YOLOv11m has the highest precision at 0.9287, mAP@50 at 0.9447, and mAP@50-95 at 0.7675 making it the most accurate. It had the second-highest recall at 0.8860, trailing only YOLOv12s. Fig. 14 shows the metrics.

YOLOv12n has a better mAP@50-95 at 0.7320 than YOLOv8n and YOLOv11n. It provides a reasonable balance with 0.8782 for precision and 0.8440 for recall. Fig. 15 shows the metrics.

YOLOv12s has the highest recall value 0.9134 compared to all other models, showing it detects more objects. However, it has the lowest precision (0.8552), indicating higher false positives. Fig. 16 shows the metrics.

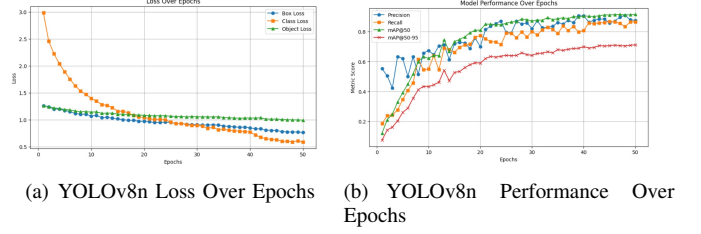


Fig. 8. YOLOv8n Training Metrics

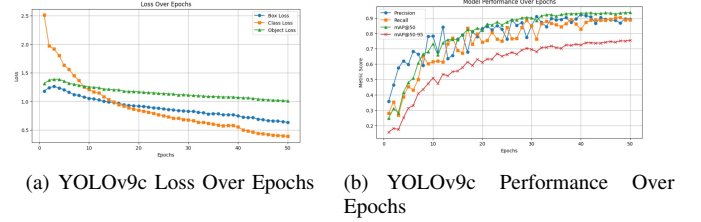


Fig. 9. YOLOv9c Training Metrics

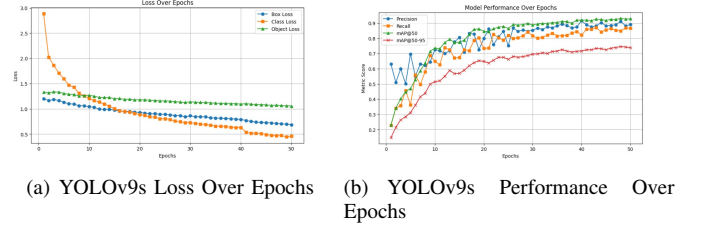


Fig. 10. YOLOv9s Training Metrics

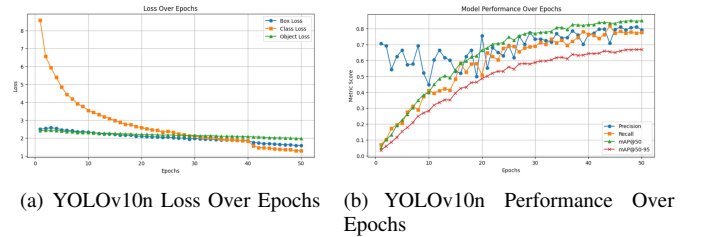


Fig. 11. YOLOv10n Training Metrics

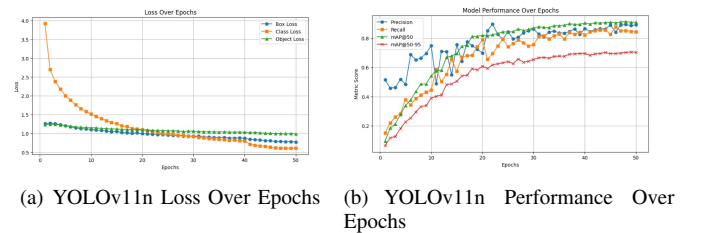
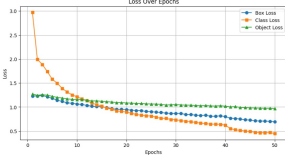
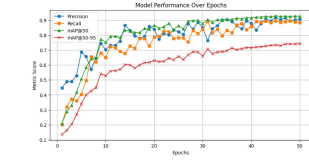


Fig. 12. YOLOv11n Training Metrics

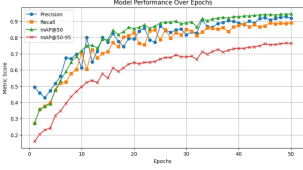


(a) YOLOv11s Loss Over Epochs

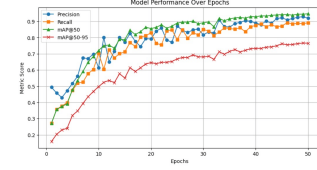


(b) YOLOv11s Performance Over Epochs

Fig. 13. YOLOv11s Training Metrics

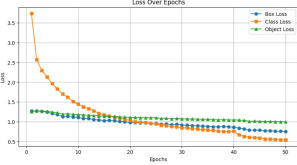


(a) YOLOv11m Loss Over Epochs

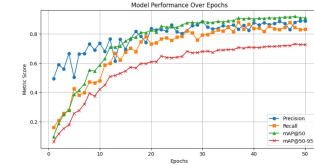


(b) YOLOv11m Performance Over Epochs

Fig. 14. YOLOv11m Training Metrics

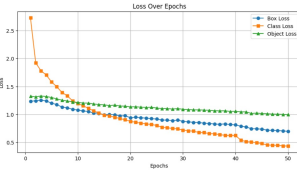


(a) YOLOv12n Loss Over Epochs

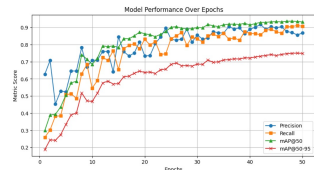


(b) YOLOv12n Performance Over Epochs

Fig. 15. YOLOv12n Training Metrics



(a) YOLOv12s Loss Over Epochs



(b) YOLOv12s Performance Over Epochs

Fig. 16. YOLOv12s Training Metrics

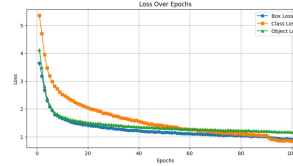
TABLE I
FINAL PERFORMANCE RESULTS OF BASE YOLO MODELS.

Model	Precision	Recall	mAP@50	mAP@50-95
YOLOv8n	0.8755	0.8654	0.9141	0.7117
YOLOv9s	0.9109	0.8490	0.9308	0.7475
YOLOv9c	0.8939	0.8874	0.9364	0.7550
YOLOv10n	0.7951	0.7782	0.8513	0.6696
YOLOv11n	0.8866	0.8472	0.9086	0.7068
YOLOv11s	0.9058	0.8842	0.9254	0.7424
YOLOv11m	0.9287	0.8860	0.9447	0.7675
YOLOv12n	0.8782	0.8440	0.9200	0.7320
YOLOv12s	0.8552	0.9134	0.9347	0.7495

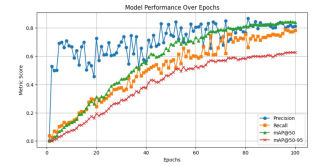
B. Performance Evaluation of Attention-Augmented Model

An analysis of the model's performance is given by a treatment of loss convergence and detection over 100 epochs of training. Fig. 17 shows how the losses collectively decreased. Classification loss, formerly greater than 5.0, considerably improved during the training, dropping to less than 1.0 in the final epochs. This reduction indicates that the spatial and channel attention from CBAM modules improved class discrimination. Box loss and objectness losses converged smoothly, signifying improved localization and greater confidence in the presence of objects.

Fig. 17 displays the progress of the model metrics within a similar period. Both precision and recall curves steadily grew and reached 0.8135 and 0.7859 by the end, respectively. Further, it attained a mAP of 0.8419 at IoU 0.5 and mAP 0.6278 throughout the range of 0.5–0.95 IoU. These scores assure the model's ability to detect and localize objects in both lenient and strict conditions. All such improvements are attributed to the CBAM modules, which enhanced attention toward spatially and semantically meaningful regions—especially helpful for detecting small or partially occluded marine debris. The evaluation metrics for the YOLOv11n model integrated with CBAM are presented in Table II.



(a) Training Loss over Epochs



(b) Detection Metrics over Epochs

Fig. 17. YOLOv11n + CBAM: Training Loss and Detection Metrics

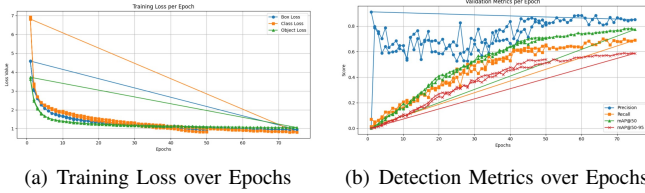
TABLE II
PERFORMANCE OF YOLOv11N WITH CBAM

Model	Precision	Recall	mAP@50	mAP@50-95
YOLOv11n+CBAM	0.8755	0.8654	0.9141	0.7117

C. Evaluation of YOLOv11n with ResNet Backbone Integration

A complete YOLOv11n model with a ResNet-like backbone was investigated to observe the way residual learning may affect detection performance. The movement in training losses (i.e., box, classification, objectness losses) across 75 epochs is shown in Fig. 18. The training losses followed smooth trajectories that consistently moved toward zero. The classification losses were initially high, exceeding values of 6.5; a greater than expected rapid convergence toward zero was evidenced after approximately 25 epochs. Values eventually stabilized just above zero (below one), indicating high accuracy on class detection. Box losses and objectness losses had smooth convergence to levels expected in testing workflows. The model successfully learned to localize and recognize the objects in the training data.

Fig. 18 shows the model validation metrics during epochs of learning. The best precision was 0.8298, whereas the best recall was 0.6471; both indicate strong consistency in correct positive detections. The model performed effectively across increasingly difficult intersection-over-union (IoU) thresholds, achieving a mean Average Precision (mAP) at IoU 0.5 of 0.7094 and a mAP@50–95 of 0.5398. These results indicate that the residual connectivity introduced to the backbone enabled the network to retain and exploit spatially informative features, which is especially beneficial in complex marine environments with fine structures. The performance results of the YOLOv11n model with a ResNet backbone are detailed in Table III.



(a) Training Loss over Epochs (b) Detection Metrics over Epochs

Fig. 18. YOLOv11n + ResNet: Training Loss and Detection Metrics

TABLE III
PERFORMANCE OF YOLOV11N WITH RESNET BACKBONE

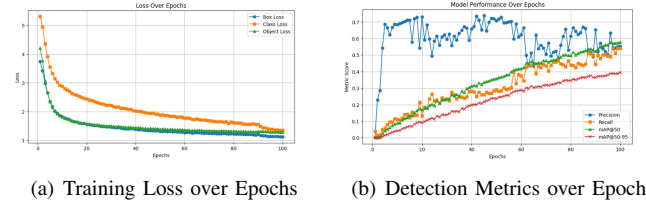
Model	Precision	Recall	mAP@50	mAP@50-95
YOLOv11n+ResNet	0.8298	0.6471	0.7094	0.5398

D. Evaluation of YOLOv11n with YOLOv12n Backbone Integration

The YOLOv11n model, with its YOLOv12n-modified backbone, was evaluated based on loss convergence and standard object detection metrics. The training loss curves, as seen in Fig. 19, consistently decreased throughout training for all components—box loss, classification loss, and objectness loss. For example, classification loss started out very high but steadily reduced as epochs progressed, indicating the model’s increasing ability to differentiate between object classes. Similarly, box and objectness losses exhibited stable convergence,

reflecting that the model retained its localization and object detection capabilities throughout training.

Fig. 19 also depicts the model’s detection performance after 100 epochs. The model attained precision, recall, mAP@0.5, and mAP@0.5:0.95 of 0.5524, 0.5406, 0.5790, and 0.3951, respectively. While these values are average compared to the attention-augmented variants, they demonstrate that the deeper and more expressive YOLOv12n backbone improved the model’s generalization capabilities over the TrashCan dataset, particularly for objects of varying scales and challenging underwater backgrounds. The evaluation metrics for the YOLOv11n model utilizing the YOLOv12n backbone are provided in Table ??.



(a) Training Loss over Epochs (b) Detection Metrics over Epochs

Fig. 19. YOLOv11n + YOLOv12n Backbone: Training Loss and Detection Metrics

TABLE IV
PERFORMANCE OF YOLOV11N WITH YOLOV12N BACKBONE

Model	Precision	Recall	mAP@50	mAP@50-95
YOLOv11n+YOLOv12n	0.5524	0.5406	0.5790	0.3951

E. Parameter Settings Table

All YOLO models in this research were trained with identical hyperparameter settings for a fair comparison, as described in Table V.

TABLE V
HYPERPARAMETER SETTINGS FOR YOLO MODELS

Model	Parameter	Value
Baseline Models	Input Size	416x416
	Batch Size	16
	Learning Rate	0.01
	Optimizer	SGD
	Weight Decay	0.0005
	Momentum	0.937
Model With Attentions	Epochs	50
	Input Size	416x416
	Batch Size	16
	Learning Rate	0.01
	Optimizer	SGD
	Weight Decay	0.0005
Model With ResNet Backbone	Momentum	0.937
	Epochs	100
	Input Size	416x416
	Batch Size	16
	Learning Rate	0.01
	Optimizer	SGD
Model With YOLOV12n Backbone	Weight Decay	0.0005
	Momentum	0.937
	Epochs	50
	Input Size	416x416
	Batch Size	16
	Learning Rate	0.01

F. Best Performing Model

Out of all the models evaluated, the baseline YOLOv11n produced the best performance for all evaluation metrics and was the most effective structure for the detection of trash under water in this study. The baseline model without extra attention modules or altered backbone achieved a precision of 0.9287, recall of 0.8860, mAP@50 of 0.9447, and mAP@50-95 of 0.7675. These metrics highlight the highly effective model as shown consistently for detection, localization, and robustness across large and small and varying levels of occlusion.

YOLOv11n's design is optimal, with a considered approach to balancing depth, feature fusion, and processing time. YOLOv11n has a backbone to extract deep features, neck layers for feature fusion (PANet style), and head layers that generate bounding boxes and class predictions. The model was trained with 416x416 input images, and the YOLOv11n was trained for 100 epochs with the same hyperparameters on the TrashCan dataset, and in all variants.

One main reason for YOLOv11n's performance is its ability for learning optimal discriminative features, without overfitting for complex underwater imagery associated with turbidity, lower contrast, and distortion. YOLOv11n's feature fusion enables the model to localize and classify large and small items of debris.

The baseline architecture YOLOv11n emerged as the most well-balanced and efficient solution in this work. Its high accuracy coupled with its capability to perform real-time inference makes it an excellent candidate to be deployed in marine surveillance systems and autonomous underwater platforms.

VI. COMPARATIVE ANALYSIS OF MODELS

A. Baseline vs Attention Model

The attention-augmented YOLOv11n model, enhanced with CBAM modules applied to the model layers 14, 18, and 22, improved all metrics in comparison to the baseline YOLOv11n model. Baseline metrics were precision of 0.7441, recall of 0.6159, mAP@50 of 0.6653, and mAP@50-95 of 0.4812. For the attention-augmented YOLOv11n model, precision was 0.8755, recall was 0.8654, mAP@50 was 0.9141, and mAP@50-95 was 0.7117. Clearly, the improvements validate the application of attention mechanisms, which offer improvements with feature refinement and object localization. Among the two methods; the attention-augmented YOLOv11n improved significantly on the baseline YOLOv11n model. The improved performance suggests spatial attention and channel of attention can improve the YOLOv11n model performance by utilizing spatial attention to focus on meaningful features, especially in underwater scene characteristics of noise, clutter and occlusions.

B. Baseline vs Backbone Variation models

Two YOLOv11n models with modified backbones were assessed against baseline. The ResNet-backbone variant met with a precision of 0.8298, recall of 0.6471, mAP@50 of 0.7094, and mAP@50-95 of 0.5398; showing substantial

changes in detection efficacy and accuracy, as well as robustness. Conversely, the YOLOv12n-backbone variant, which had a lower total performance, still demonstrated moderate improvements over baseline in generalization. Its precision was 0.5524, recall was 0.5406, mAP@50 was 0.5790, and mAP@50-95 was 0.3951. The YOLOv12n-backbone variant gained an advantage from the newer and deeper architecture of YOLOv12n, as it profited from structural depth advancements and modified convolutional strategies. Both backbone variants improved performance beyond the original YOLOv11n model. Comparing the improvements, the ResNet-backbone variant was superior to the YOLOv12n-backbone in all metrics, indicating that the explicit residual connections in the ResNet architecture provided greater gradient flow, with more efficient feature propagation, and as a result, more detection efficacy on better identifying small and complex marine debris. The YOLOv12n-backbone provided architectural developments from a younger generation; however, it did not provide efficacy as per enhancements equivalent to the performance improvements provided by ResNet for this specific application context.

VII. CONCLUSION AND FUTURE SCOPE

In this study we trained and evaluated a total of 11 YOLO-based object detection models for the task of underwater marine debris detection using the TrashCan dataset which consisted of annotated frames derived from JAMSTEC deep-sea footage. These models consisted of standard baseline architectures, attention-augmented versions with CBAM, and versions with their backbone modified with ResNet+BestYOLOn or YOLOv12n integrations.

Of all configurations, the baseline YOLOv11n model produced the best overall performance with a mAP@0.5 of 0.9447, precision of 0.9287, and recall of 0.8860. Although attention mechanisms improved feature refinement and backbone modifications increased architectural variability, these enhancements did not always provide statistically significant detections greater than the baseline model. This suggests that baseline models that are well-tuned and without added complexity can still deliver strong outcomes in specific domains, like underwater object detection.

Future work could develop the ability to apply temporal modeling across video frames and utilize transformer-based detection heads. Implementing semi-supervised learning could also improve performance and generalizable object detection with less data to train the model. Furthermore, transitioning the system to work on edge devices or autonomous underwater vehicles (AUVs) could facilitate a position for real-time monitoring of marine litter and provide scale for these actions in an environmental protection capacity.

VIII. REFERENCES

- [1] F. Rehman, M. Rehman, M. Anjum, and A. Hussain, "Optimized YOLOv8: An efficient underwater litter detection using deep learning," *Ain Shams Engineering Journal*, vol. 16, no. 1, p. 103227, Jan. 2025, doi: 10.1016/j.asej.2024.103227.
- [2] B. Jalil, L. Maggiani, and L. Valcarengi, "Optimized Monitoring of Underwater Marine Waste Using Knowledge Distillation," in 2024 IEEE

- International Workshop on Metrology for the Sea; Learning to Measure Sea Health Parameters (MetroSea), IEEE, Oct. 2024, pp. 78–82. doi: 10.1109/MetroSea62823.2024.10765682.
- [3] F. Zhao et al., “Riverbed litter monitoring using consumer-grade aerial-aquatic speedy scanner (AASS) and deep learning based super-resolution reconstruction and detection network,” *Mar Pollut Bull*, vol. 209, p. 117030, Dec. 2024, doi: 10.1016/j.marpolbul.2024.117030.
- [4] N. H. Krishna and R. P. Kumar, “Advancing Marine Plastic Detection with CLAHE-Enhanced Underwater Images Using YOLOv8 Integrated Attention Mechanism,” 2024, pp. 277–292. doi: 10.1007/978-981-97-3442-9_19.
- [5] B. Jalil, L. Maggiani, and L. Valcarengi, “Convolutional Neural Networks and Transformers-Based Techniques for Underwater Marine Debris Classification: A Comparative Study,” *IEEE Journal of Oceanic Engineering*, pp. 1–14, 2024, doi: 10.1109/JOE.2024.3436769.
- [6] S. Saji, M. S. Manikandan, J. Zhou, and L. R. Cenkeramaddi, “Underwater Debris Detection Using Visual Images and YOLOv8n for Marine Pollution Monitoring,” in 2024 IEEE 19th Conference on Industrial Electronics and Applications (ICIEA), IEEE, Aug. 2024, pp. 1–6. doi: 10.1109/ICIEA61579.2024.10664718.
- [7] P. Pachaiyappan, G. Chidambaram, A. Jahid, and M. H. Alsharif, “Enhancing Underwater Object Detection and Classification Using Advanced Imaging Techniques: A Novel Approach with Diffusion Models,” *Sustainability*, vol. 16, no. 17, p. 7488, Aug. 2024, doi: 10.3390/su16177488.
- [8] R. Jain, S. Zaware, N. Kacholia, H. Bhalala, and O. Jagtap, “Advancing Underwater Trash Detection: Harnessing Mask R-CNN, YOLOv8, EfficientDet-D0 and YOLACT,” in 2024 2nd International Conference on Sustainable Computing and Smart Systems (ICSCSS), IEEE, Jul. 2024, pp. 1314–1325. doi: 10.1109/ICSCSS60660.2024.10625362.
- [9] B. C. Corrigan, Z. Y. Tay, and D. Konovessis, “Real-Time Instance Segmentation for Detection of Underwater Litter as a Plastic Source,” *J Mar Sci Eng*, vol. 11, no. 8, p. 1532, Jul. 2023, doi: 10.3390/jmse11081532.
- [10] L. Jiang, F. Liu, J. Lv, B. Liu, and C. Wang, “GST-YOLO: a lightweight visual detection algorithm for underwater garbage detection,” *J Real Time Image Process*, vol. 21, no. 4, p. 114, Aug. 2024, doi: 10.1007/s11554-024-01494-w.
- [11] A. Khriiss, A. Kerkour Elmiad, M. Badaoui, A.-E. Barkaoui, and Y. Zarhloule, “Exploring Deep Learning for Underwater Plastic Debris Detection and Monitoring,” *Journal of Ecological Engineering*, vol. 25, no. 7, pp. 58–69, Jul. 2024, doi: 10.12911/22998993/187970.
- [12] Singh Walia, J. and K, P. L., “Deep Learning Innovations for Underwater Waste Detection: An In-Depth Analysis”, *arXiv e-prints*, Art. no. arXiv:2405.18299, 2024. doi:10.48550/arXiv.2405.18299.
- [13] J. Zhu, T. Hu, L. Zheng, N. Zhou, H. Ge, and Z. Hong, “YOLOv8-C2f-Faster-EMA: An Improved Underwater Trash Detection Model Based on YOLOv8,” *Sensors*, vol. 24, no. 8, p. 2483, Apr. 2024, doi: 10.3390/s24082483.
- [14] X. Zhang, D. Zhu, and W. Gan, “YOLOv7t-CEBC Network for Underwater Litter Detection,” *J Mar Sci Eng*, vol. 12, no. 4, p. 524, Mar. 2024, doi: 10.3390/jmse12040524.
- [15] C. Gupta, N. S. Gill, P. Gulia, S. Yadav, and J. M. Chatterjee, “A novel finetuned YOLOv8 model for real-time underwater trash detection,” *J Real Time Image Process*, vol. 21, no. 2, p. 48, Apr. 2024, doi: 10.1007/s11554-024-01439-3.
- [16] K. Demir and O. Yaman, “Projector deep feature extraction-based garbage image classification model using underwater images,” *Multimed Tools Appl*, vol. 83, no. 33, pp. 79437–79451, Mar. 2024, doi: 10.1007/s11042-024-18731-w.
- [17] A. A. Alsuwaylimi, “Enhanced YOLOv8-Seg Instance Segmentation for Real-Time Submerged Debris Detection,” *IEEE Access*, vol. 12, pp. 117833–117849, 2024, doi: 10.1109/ACCESS.2024.3448258.
- [18] W. Jiang, L. Yang, and Y. Bu, “Research on the Identification and Classification of Marine Debris Based on Improved YOLOv8,” *J Mar Sci Eng*, vol. 12, no. 10, p. 1748, Oct. 2024, doi: 10.3390/jmse12101748.
- [19] A. Sánchez-Ferrer, J. J. Valero-Mas, A. J. Gallego, and J. Calvo-Zaragoza, “An experimental study on marine debris location and recognition using object detection,” *Pattern Recognit Lett*, vol. 168, pp. 154–161, Apr. 2023, doi: 10.1016/j.patrec.2022.12.019.
- [20] H. Deng, D. Ergu, F. Liu, B. Ma, and Y. Cai, “An Embeddable Algorithm for Automatic Garbage Detection Based on Complex Marine Environment,” *Sensors*, vol. 21, no. 19, p. 6391, Sep. 2021, doi: 10.3390/s21196391.